

Backoffice UI — Angular Version Migration — Project Interview Pack

Target: 5-year Full-Stack Developer (Angular + Java) — project-understanding round **Idea:** Every answer is tied to the **real Backoffice migration project** — so when an interviewer asks *"in the Angular migration you mentioned, what challenges did you face?"*, you don't freeze. You have a strategy, concrete breaking changes, code, and stories.

How to use this pack

- Each file = one interview angle. Read the **Question** → **Answer** structure and say answers **out loud**.
- Memorize the *flow and the reasons*, not the code character-for-character. Interviewers reward "why", not recitation.
- The one file that wins this round is `01-migration-strategy-deep-dive.md` — that is exactly where interviewers push. Know it cold.
- 30 minutes before an interview, read **only** `00-ONE-PAGE-CHEATSHEET.md`.

The sections

#	File	Covers
0	<code>00-ONE-PAGE-CHEATSHEET.md</code>	20 must-know facts + one-liner answers for a 30-min pre-interview glance
1	<code>01-migration-strategy-deep-dive.md</code>	THE core file — why never jump majors, incremental <code>ng update</code> , the <code>update.angular.io</code> checklist, build+test+commit-per-version discipline, bisecting, rollback, driving four repos
2	<code>02-breaking-changes-per-version.md</code>	Version-by-version reference of the real breaking changes 2→16 and 9/4→16: Ivy, RxJS 6→7, <code>entryComponents</code> , <code>ViewChild</code> static flag, differential loading, TS bumps, strict mode, <code>angular.json</code> schema, standalone/signals
3	<code>03-angular-internals.md</code>	What interviewers probe AFTER a migration answer: View Engine vs Ivy, change detection, zone.js, RxJS operators, lifecycle hooks, modules vs standalone — anchored to "this is what I had to understand to migrate safely"
4	<code>04-testing-and-stars.md</code>	Seamless-transition testing strategy + 3-4 STAR stories (RxJS 6→7, Ivy-incompatible library, SSG-plugin-in-host, "how did you make it seamless") + Git workflow + "what I'd do differently"

Key project facts (know these cold)

- **Backoffice UI Angular Version Migration** — **Feb 2024 – Mar 2024** (focused ~2-month effort).
- Migrated **four repositories to Angular 16**:
- **1 Ticketing web application** — **Angular 2 → 16**.

- **3 SSG web-plugin app repos — Angular 9 and 4 → 16.**
- **Goal:** navigate years of breaking changes safely, verify nothing regressed via meticulous testing.
- **Payoff:** modern, supported, faster-building apps on **one** Angular version.
- **My role:** I **led and owned** the migrations across all four repos; QA validated. I was the person who understood the cross-version breaking changes and applied them consistently.
- **Core method:** never jump majors — **one major at a time** (`ng update` per step), **build + test after every step, one commit per version** so I could bisect and roll back. `update.angular.io` was my per-step checklist.
- **Headline breaking changes I handled:** Ivy default renderer (v9), **RxJS 6→7** (`toPromise()` → `firstValueFrom` / `lastValueFrom` , stricter typing), removed/deprecated APIs (`entryComponents` , `ViewChild` `static` flag), TypeScript strictness bumps, `angular.json` build-schema changes, third-party-library Ivy compatibility, and **SSG plugins loading inside a host environment**.

Honesty note (important — read this)

This was a **focused ~2-month migration project, not a from-scratch product build**. Do not oversell it as a huge greenfield app. Frame it as what it actually was and what it actually proves:

"It was a focused migration — but a deep one. To take a repo from Angular 2 to 16 safely you have to understand Angular internals across every major version: the View Engine→Ivy switch, how change detection and zone.js work, RxJS's evolution, and how the build pipeline changed. My value was the upgrade strategy and the Angular-internals knowledge that let me do it without regressions across four repos."

That framing is a strength: it signals **deep Angular-internals expertise + disciplined upgrade strategy + risk management**, which is exactly what a 5-year Angular engineer should own. Interviewers respect "here's the focused scope + here's the depth it required" far more than an inflated story you can't defend.

00 — One-Page Cheat Sheet (read 30 min before the interview)

Project: Backoffice UI Angular Version Migration · Feb–Mar 2024 · I led/owned it, QA validated. **Scope:** 4 repos → **Angular 16**. 1 Ticketing app (**Angular 2** → **16**) + 3 SSG web-plugin apps (**Angular 9** and **4** → **16**). **Goal:** cross years of breaking changes safely, prove nothing regressed. **Payoff:** modern, supported, faster-building apps on one version.

The 20 facts + one-liner answers

1. **What was the project?** "Migrating four Backoffice repos to Angular 16 — a Ticketing app from Angular 2, and three SSG plugin apps from Angular 9 and 4."
2. **Your role?** "I led and owned the migrations across all four repos; QA regression-tested each app."
3. **Biggest strategic rule?** "Never jump majors. Angular only supports **one-major-at-a-time** upgrades, so I went 9→10→11...→16."
4. **The per-step tool?** "`ng update @angular/core@N @angular/cli@N` — it runs the migration **schematics** that auto-fix a lot of code."
5. **Your checklist?** "`update.angular.io` — you pick from-version and to-version and it lists every change per step."
6. **Discipline per step?** "**Build + run tests after every single version hop**, not just at the end — it localizes any breakage to one hop."
7. **Git discipline?** "**One commit per version**, branch per repo, PR to the owning team. Granular commits = documentation + I can `git bisect` and roll back one step."
8. **Ivy (v9)?** "Ivy became the **default renderer** in v9 — smaller bundles, better tree-shaking, faster builds. Main risk: libraries built for the old **View Engine** had to be Ivy-compatible."
9. **RxJS 6→7?** "Stricter typing and `toPromise()` **deprecated** → I moved to `firstValueFrom / lastValueFrom`."
10. `toPromise()` **fix (say the code):** "`await firstValueFrom(this.http.get(...))` instead of `await this.http.get(...).toPromise()`."
11. `entryComponents` ? "Ivy made it unnecessary — dynamic components no longer need pre-declaration, so I removed the `entryComponents` arrays (fully removed in v13)."
12. `ViewChild` **static flag (v8/9)?** "v8 made `{ static: true|false }` **required**; v9 defaulted it to `false`. I set `static: true` only where I read the ref in `ngOnInit`."
13. **Differential loading (v8)?** "v8 shipped modern + legacy bundles via `browserslist / tsconfig` targets; later removed as browsers modernized — just meant build-config changes."
14. **TypeScript bumps?** "Each major requires a newer TS. Stricter compiler + **strict templates** surfaced previously-silent type errors I had to fix."
15. `angular.json` **schema?** "The build schema changed across versions (builder targets, `polyfills` moved to an array, budgets); `ng update` migrated most of it, I reconciled the rest."
16. **Third-party libs?** "I checked each dependency's Angular compatibility **before each hop** — upgrade it, swap for a maintained alternative, or pin-and-isolate. Ivy compat was the big filter at v9."

17. **SSG plugins — special?** "They're **embedded plugins loaded inside a host SSG environment**, so I had to verify the upgraded bundles still **bootstrap/mount** correctly in the host and don't clash with its runtime — testing the integration boundary mattered as much as in-app behavior."
18. **How 'seamless'?** "Build+test each step, smoke-test critical flows, QA full regression, and I added test coverage to risky areas **before** upgrading them."
19. **Standalone components / signals?** "Standalone stabilized in v15, signals arrived in v16. We landed on 16 as NgModule apps; I know how to adopt standalone/signals as the next step."
20. **What you'd do differently?** "Strengthen automated tests **before** starting, and script the per-step build/test loop in **CI** so each bump is gated automatically."

Version map (memorize the milestones, not every number)

- **v8** — `ViewChild` static flag required · differential loading introduced.
- **v9** — **Ivy default** · `entryComponents` deprecated · static flag defaults to false.
- **RxJS 7** (with Angular ~12) — `toPromise()` deprecated · stricter types.
- **v12** — strict mode default for new projects · Ivy-only (View Engine deprecated for libs).
- **v13** — View Engine removed · IE11 dropped · `entryComponents` removed.
- **v14** — standalone components (preview) · typed reactive forms.
- **v15** — **standalone stable** · `NgOptimizedImage`.
- **v16** — **signals (preview)** · `takeUntilDestroyed()` · required inputs · esbuild dev server (preview).

If you freeze, say this first

"It was a focused migration across four repos to Angular 16. The core challenge was doing multi-version jumps safely — I never jumped majors, I upgraded one major at a time with `ng update`, built and tested after every step, and kept one commit per version so I could roll back. The specific breaking changes I handled were Ivy at v9, the RxJS 6-to-7 migration, and removed APIs like `entryComponents`."

Then let them pick a thread. You have a file for each.

01 — Migration Strategy Deep Dive (the core file)

This is where interviewers push. When they say "*what challenges did you face in the Angular migration?*", they are really asking "*do you understand how to do a risky, multi-version upgrade without breaking production?*" Everything below is your answer.

Context: 4 Backoffice repos → Angular 16. Ticketing app **Angular 2 → 16**, three SSG plugin apps **Angular 9 and 4 → 16**. I led/owned it; QA validated.

Q1. What was your overall strategy for a jump as big as Angular 2/9 to 16?

Answer. The single most important rule: **I never jump majors.** Angular only officially supports **one-major-at-a-time** upgrades — each major ships **migration schematics** that assume you're coming from the immediately previous major. If you skip versions, those schematics don't exist for your jump and you're hand-fixing years of changes at once with no safety net.

So the strategy was a disciplined loop, repeated once per major:

```
for each major version step (e.g. 9 → 10 → 11 → ... → 16):
  1. Read the update.angular.io checklist for THIS step
  2. Check third-party libs are compatible with the target major
  3. ng update @angular/core@N @angular/cli@N    # runs the schematics
  4. Fix whatever the schematics couldn't auto-migrate
  5. Build                → must be green
  6. Run unit tests       → must be green
  7. Smoke-test critical flows
  8. Commit: "chore: upgrade to Angular N"        # one commit per version
```

The value of the loop is **localization of risk**: if something breaks, it broke in exactly one hop, against one commit, and I can bisect to it. A big-bang "2 → 16 in one branch" would have been an undebuggable mess.

Q2. Why is jumping majors actually dangerous — what's the mechanism?

Answer. Three concrete reasons, not just "best practice":

1. **Schematics are per-step.** `ng update` runs code-mod migrations shipped *inside* each major, written to migrate from `N-1` to `N`. Angular 12's schematics know how to fix Angular-11 code, not Angular-4 code. Skip versions and you lose all automation.
2. **Peer-dependency math.** Each Angular major pins a compatible **TypeScript**, **RxJS**, **zone.js**, and Node range. Jumping straight to 16 forces TS/RxJS to their newest at once, so *every* type error and deprecation lands simultaneously with no way to tell which version introduced it.
3. **Debuggability.** With one-major-at-a-time, a regression is bounded by a single hop's changelog. With a big jump, a bug could come from any of a dozen releases.

I say it plainly in interviews: *"Incremental isn't ceremony — it's the only way to keep each failure attributable to one change."*

Q3. Walk me through the `ng update` step. What does it actually do?

Answer. `ng update` is more than a package bump. For each step:

```
# check what's upgradable and to what version
ng update

# perform ONE major step (core + cli together, matched versions)
ng update @angular/core@12 @angular/cli@12
```

It does four things: 1. Bumps `@angular/*` and peer deps (TypeScript, RxJS, zone.js) to versions compatible with that major. 2. Runs **migration schematics** — automated code-mods. Examples I hit: the `ViewChild / ContentChild` static-flag migration, the RxJS-6→7 `toPromise` deprecation flags, the `entryComponents` removal, and `angular.json` schema updates. 3. Updates `angular.json`, `tsconfig`, and `polyfills` to the new expected shape. 4. Refuses to run on a dirty git tree — which is *why* the commit-per-version discipline pairs naturally with it.

The schematics do maybe 70-80% of the mechanical work. The remaining 20-30% — third-party incompatibilities, custom code the schematic can't reason about, template type errors — is the human part, and that's what I owned.

Q4. What was your per-step checklist? How did you know what would break?

Answer. The **Angular Update Guide** at `update.angular.io` was my checklist. You select "from version", "to version", app complexity (basic/medium/advanced), and it emits a categorized list — **before you begin, during the update, after the update** — of every required and optional change for that specific hop. I ran it fresh for **each** step, not once for the whole journey.

Around that I kept a small manual checklist per repo: - Dependency compatibility matrix (does each 3rd-party lib support the target major / is it Ivy-compatible?). - Deprecation warnings from the previous build (fix them *before* they become removals). - Known risky areas in that app (where tests were thin — I added coverage first).

Q5. Why one commit per version? Isn't that a lot of commits?

Answer. Yes, and that's the point. **One commit per major version** gave me three things:

1. `git bisect` — if a regression surfaced only after several hops, I could bisect across the version commits to find exactly which major introduced it.

2. **One-step rollback** — if v13 caused a problem I couldn't immediately solve, I could revert to the v12 commit and keep the app shippable while I investigated, instead of losing the whole migration.
3. **Documentation for free** — each commit message ("upgrade to Angular 13: remove View Engine refs, drop entryComponents") is a precise record of what that version required. The owning team reviewing the PR could read the history hop-by-hop.

```
* chore: upgrade to Angular 16 (signals-ready, takeUntilDestroyed)
* chore: upgrade to Angular 15 (standalone available, NgOptimizedImage)
* chore: upgrade to Angular 14 (typed reactive forms)
* chore: upgrade to Angular 13 (remove View Engine, drop entryComponents)
* chore: upgrade to Angular 12 (RxJS 7, strict mode, Ivy-only libs)
* chore: upgrade to Angular 11
* chore: upgrade to Angular 10
(base: Angular 9)
```

Q6. How did you handle the fact that it was FOUR repos at different start versions?

Answer. I treated each repo as its own migration with its own branch, but reused one **playbook** across all four:

- **Ticketing app: Angular 2 → 16.** The longest chain. Note that Angular 2→4 is a valid `ng update` sequence (there was no Angular 3 — they skipped it to align router versioning), then 4→5→6...→16. The early hops (2→4→5) were the roughest because tooling was less mature and the CLI itself changed a lot (e.g. the move to `angular.json` from `.angular-cli.json` at v6).
- **Three SSG plugin apps: two starting at 9, one at 4.** The two on 9 already had Ivy as an option and were shorter chains; the one on 4 followed the long path like Ticketing.

Because I did them one after another, later repos were faster — I'd already solved the shared breaking changes (RxJS, Ivy libs, `angular.json` schema) once, so I applied the known fix immediately. Doing them sequentially rather than in parallel was deliberate: I built the playbook on the first repo and reused it.

Q7. The v6 jump is famously awkward (`.angular-cli.json` → `angular.json`). How did that go?

Answer. Angular 6 restructured the workspace: `.angular-cli.json` became `angular.json` with a completely new **architect/builder** schema, and `ng update` / `ng add` were introduced. The v6 schematic migrated the config file automatically, but I verified the generated `angular.json` by hand — build options, `assets`, `styles`, `scripts`, and environment `fileReplacements` — because a mis-migrated builder config fails in ways that look like app bugs. This is exactly why "build after every step" matters: the v6 config change is caught by the very next `ng build`.

Q8. What did the build/test checkpoint actually catch?

Answer. Real examples across the hops: - **Build caught:** template type errors newly flagged by strict templates; removed API references (`entryComponents`, `Renderer` → `Renderer2`); `angular.json` schema mismatches; TS syntax the newer compiler rejected. - **Unit tests caught:** RxJS typing/behavior changes (e.g. `toPromise` semantics, stricter operator overloads); DI changes; components whose lifecycle timing shifted with Ivy. - **Smoke tests caught:** runtime issues that compile fine but behave differently — the SSG plugins mounting inside the host, and change-detection timing differences under Ivy.

The rule I repeat: "a green build and green tests after every hop is the difference between a controlled migration and a debugging nightmare at the end."

Q9. What was your rollback plan if a version step went wrong?

Answer. Layered: 1. **Never merge a broken hop.** Each version lived on the migration branch and only advanced once build+test+smoke were green. 2. **One-commit revert.** Because each version was its own commit, I could `git revert` /reset back exactly one version and the app was shippable again on the previous major. 3. **Isolate, don't block.** If a single third-party library blocked a hop, I'd pin/isolate that library and continue rather than abandon the whole step (see the Ivy-library story in 04). 4. **The owning team's PR review** was a final gate before anything hit their main branch.

Q10. How long did it take, and how did you keep it from dragging?

Answer. It was a focused ~2-month effort (Feb–Mar 2024). What kept it moving: - The **playbook reuse** across repos — I wasn't re-learning each breaking change. - `ng update` **schematics** doing the bulk mechanical work. - **Doing the shortest-chain repos and cheapest fixes first** to build momentum and confidence, then the long Angular-2 Ticketing chain. - Not gold-plating: the goal was "safely on 16, nothing regressed," not "rewrite to standalone/signals." I landed the apps as **NgModule apps on 16** and noted standalone/signals as the *next* project.

The 30-second version (if they just want the headline)

"Four repos to Angular 16. Rule number one: never jump majors — Angular's migration schematics are per-step, so I went one major at a time with `ng update`, ran the `update.angular.io` checklist each hop, and built + tested after every single step. I committed one version per commit so any regression was attributable to one hop and I could bisect or roll back. The hard parts were Ivy at v9, the RxJS 6-to-7 migration, removed APIs like `entryComponents`, and making sure the three embedded SSG plugins still mounted inside their host environment."

02 — Breaking Changes, Version by Version (reference)

The real breaking changes I crossed going **Angular 2** → **16** (Ticketing) and **9/4** → **16** (SSG plugins). For each: **what it was**, **why it broke**, **how I fixed it**. Interviewers love when you can name the version a change landed in — it proves you lived it, not read a blog.

Mental model: the migration wasn't one big change, it was ~14 small, well-documented steps. Below are the ones that actually cost me time.

Angular 2 → 6 (the early, awkward hops — Ticketing app)

Angular 2 → 4 (no Angular 3)

- **What:** Angular skipped 3 to align the router's version with the rest of the framework. `ng update` treats 2→4 as one step.
- **Why it broke:** template `<template>` tag deprecated in favor of `<ng-template>`; some animation imports moved to `@angular/animations`.
- **Fix:** ran the schematic, replaced `<template>` with `<ng-template>`, moved animation imports.

Angular 5

- **What:** build optimizer + AOT-by-default direction; `HttpModule` deprecated in favor of `HttpClientModule`; the old `Http` service replaced by `HttpClient`.
- **Why it broke:** `HttpClient` returns parsed JSON directly (no `.json()` call), and interceptors work differently.
- **Fix:** migrated `Http` → `HttpClient`, removed `.map(res => res.json())` calls, moved cross-cutting logic into `HttpInterceptor`s.

Angular 6 — the workspace restructure

- **What:** `.angular-cli.json` → `angular.json` (new architect/builder schema); `ng update` and `ng add` introduced; RxJS 5 → 6 with `rxjs-compat`; `providedIn: 'root'` tree-shakable providers.
- **Why it broke:** RxJS 6 moved to **pipeable operators** and new import paths (`rxjs/operators`), and prototype-patched operators (`.map`, `.filter`) were gone.
- **Fix:** ran the RxJS migration, converted chained operators to `.pipe(map(), filter())`, updated imports, and dropped `rxjs-compat` once clean. Hand-verified the generated `angular.json`.

```
// before (RxJS 5, patched prototype)
import 'rxjs/add/operator/map';
obs.map(x => x * 2).filter(x => x > 0);

// after (RxJS 6, pipeable)
import { map, filter } from 'rxjs/operators';
obs.pipe(map(x => x * 2), filter(x => x > 0));
```

Angular 7 → 8

Angular 8 — `ViewChild` / `ContentChild` static flag

- **What:** the `static` resolver flag became required on `@ViewChild` / `@ContentChild` queries.
- **Why it broke:** Angular needed to know whether to resolve the query *before* change detection (`static: true`, available in `ngOnInit`) or *after* (`static: false`, available in `ngAfterViewInit`). Ambiguity had to be made explicit.
- **Fix:** the v8 schematic added the flag automatically; I audited each one — `static: true` only where I actually read the ref in `ngOnInit`, otherwise `static: false`.

```
// v8 required the flag explicitly
@ViewChild('chart', { static: false }) chart!: ElementRef; // read in ngAfterViewInit
@ViewChild('form', { static: true }) form!: NgForm;        // read in ngOnInit
```

Angular 8 — differential loading

- **What:** the CLI began emitting **two bundle sets** — modern ES2015+ and legacy ES5 — chosen per browser via `browserslist` + `tsconfig target`.
- **Why it broke:** nothing broke functionally, but build config and output changed; later (as IE died) this was removed again.
- **Fix:** confirmed `browserslist` / `tsconfig.json` targets were sane for the Backoffice's supported browsers; mostly a build-config verification.

Angular 9 — the big one: Ivy

Ivy becomes the default renderer

- **What:** Ivy replaced **View Engine** as the default compilation/rendering pipeline in v9.
- **Why it broke:** Ivy = better tree-shaking, smaller bundles, faster builds, better debugging — but **libraries compiled for View Engine** (or shipping non-Ivy-compatible metadata) could fail. Ivy also changed some **change-detection timing** and made a few previously-tolerated patterns error.
- **Fix:** made sure **every dependency was Ivy-compatible** before the hop (upgrade / replace / isolate — see 04). Ran `ngcc` (the Angular compatibility compiler) which converts View-Engine libs to Ivy at install time. Fixed the handful of components whose CD timing shifted.

`entryComponents` deprecated

- **What:** with Ivy, dynamically-created components no longer need to be pre-declared, so `entryComponents` became unnecessary (deprecated v9, removed v13).
- **Why it broke:** the array was now dead code, and keeping it produced deprecation warnings.
- **Fix:** removed `entryComponents` arrays; dynamic component creation via `ViewContainerRef.createComponent()` just works under Ivy.

```
// before (View Engine) – had to pre-register
@NgModule({ entryComponents: [DialogComponent] })
// after (Ivy) – not needed; createComponent resolves it directly
this.vcr.createComponent(DialogComponent);
```

`ViewChild` static defaults to false

- **What:** in v9 the `static` flag became **optional again, defaulting to `false`**.
- **Fix:** removed the now-redundant `{ static: false }` I'd added in v8, kept `{ static: true }` only where needed.

Angular 10 → 12

Angular 10

- **What:** stricter project setup option; `browserslist` config consolidation; some TS `strict` warnings.
- **Fix:** mechanical config alignment; fixed newly-surfaced type warnings.

Angular 11 → 12 — RxJS 6 → 7

- **What:** Angular moved to **RxJS 7** (default around v12). `toPromise()` **deprecated**; **stricter typing** on operator overloads; `TimeoutError` / `combineLatest` signature tweaks.
- **Why it broke:** `toPromise()` had ambiguous semantics on empty completion, so it was deprecated in favor of `firstValueFrom` / `lastValueFrom`. Tighter overloads flagged loosely-typed `map` / `reduce` chains.
- **Fix:** replaced every `toPromise()` and added explicit types where overloads got strict.

```
// before (RxJS 6, deprecated)
const data = await this.http.get('/api/tickets').toPromise();

// after (RxJS 7)
import { firstValueFrom } from 'rxjs';
const data = await firstValueFrom(this.http.get('/api/tickets'));
// use lastValueFrom when you want the final emitted value of a finite stream
```

Angular 12 — strict mode + Ivy-only

- **What:** **strict mode on by default** for new projects; **View Engine deprecated for libraries** (Ivy-only direction); production build by default; `@angular/flex-layout` and similar started aging out.
- **Why it broke:** stricter TS/templates surfaced null-safety and type errors that were previously silent.
- **Fix:** progressively tightened `tsconfig` (`strict`, `strictTemplates`), fixed null checks and template type mismatches. I turned strict flags on **incrementally** rather than all at once so the error volume stayed manageable.

```
// tsconfig.json – tightened over several hops, not all at once
{
  "compilerOptions": { "strict": true },
  "angularCompilerOptions": { "strictTemplates": true }
}
```

Angular 13 — View Engine removed

- **What: View Engine deleted entirely** (Ivy-only); **IE11 support dropped**; `entryComponents` removed; libraries must ship Ivy (Partial Ivy / `ngcc` still bridged older ones); Node/TS bumps.
- **Why it broke:** any lingering View-Engine-only dependency or `entryComponents` reference now hard-failed.
- **Fix:** by the time I hit 13 I'd already Ivy-cleared the dependencies at v9-12 and removed `entryComponents`, so this hop was mostly confirming no View-Engine-only lib remained. Dropped IE11 from `browserslist`.

Angular 14 → 16 (the modern era)

Angular 14

- **What: standalone components (preview); typed reactive forms** (`FormControl<T>`); `inject()` function; optional `NgModules`.
- **Why it broke:** typed forms changed `FormGroup` / `FormControl` generics — untyped usages needed types or the `Untyped*` shims.
- **Fix:** left forms as `UntypedFormGroup` where a full retype was risky, typed the simpler ones. Did **not** rewrite to standalone — noted it as future work.

Angular 15

- **What: standalone components stable**; `NgOptimizedImage`; MDC-based Angular Material; router/guards can be plain functions.
- **Fix:** verified Material components still rendered after the MDC swap (some CSS/DOM differences); no forced standalone rewrite.

Angular 16 — our destination

- **What: signals (developer preview)**; `takeUntilDestroyed()` and `DestroyRef` (`@angular/core/rxjs-interop`); **required inputs**; `@angular/cli` **esbuild-based dev server** (preview); non-destructive hydration for SSR.
- **Why it mattered:** this is the modern, supported baseline we wanted — faster builds, and the door open to signals/standalone next.
- **Fix / landing state:** apps landed as **NgModule apps on Angular 16**, RxJS 7, strict where feasible. I flagged **standalone migration + signals + `takeUntilDestroyed()`** (to drop the `destroy$` Subject boilerplate) as the recommended next step.

```
// Angular 16 – the cleanup I recommended as follow-up work
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

constructor() {
  this.service.stream$
    .pipe(takeUntilDestroyed()) // no more manual destroy$ Subject
    .subscribe(v => this.handle(v));
}
```

One-line recap table (drill these)

Version	Headline change	Why it broke	How I fixed it
2→4	<code><template></code> → <code><ng-template></code>	tag deprecated	schematic + replace
5	<code>Http</code> → <code>HttpClient</code>	new API, no <code>.json()</code>	migrate calls + interceptors
6	<code>angular.json</code> + RxJS 5→6	pipeable operators, new imports	<code>.pipe()</code> , RxJS migration, verify config
8	<code>ViewChild</code> static flag required	CD-timing ambiguity	set <code>static</code> true/false per use
8	differential loading	build output changed	verify browserslist/target
9	Ivy default	View-Engine libs incompatible	Ivy-clear deps, ngcc
9	<code>entryComponents</code> deprecated	Ivy makes it dead code	remove arrays
12	RxJS 6→7, strict mode	<code>toPromise</code> deprecated, tighter types	<code>firstValueFrom</code> , add types, tighten tsconfig incrementally
13	View Engine removed, IE11 dropped	no VE fallback	confirm all deps Ivy, drop IE11
14	typed forms, standalone preview	form generics changed	type or <code>Untyped*</code> shims
15	standalone stable, Material MDC	Material DOM/CSS changed	verify UI
16	signals preview, <code>takeUntilDestroyed</code>	(destination)	land on 16, flag next steps

03 — Angular Internals (what they probe AFTER the migration answer)

Once you give a solid migration answer, interviewers test whether you *actually understand the internals* the migration touched — "you mentioned Ivy, what is it?", "you mentioned change detection, how does it work?". This file is those follow-ups. Every answer is anchored to "*this is what I had to understand to migrate safely.*"

Q1. View Engine vs Ivy — what actually changed in v9?

Answer. They're two generations of Angular's **compilation + rendering pipeline**.

- **View Engine** (pre-v9) compiled components into a shared, monolithic-ish factory format. Downsides: larger bundles, weaker tree-shaking, harder-to-read errors, and the compiler had global knowledge requirements.
- **Ivy** (default v9) compiles each component into **self-contained, locally-scoped instructions**. Benefits:
 - **Better tree-shaking** — unused Angular features get dropped, so **smaller bundles**.
 - **Faster incremental builds** — locality means recompiling one component doesn't need the whole world.
 - **Better debugging** — readable stack traces, `ng.` debug APIs in the console.
 - **AOT everywhere** — Ivy made **Ahead-of-Time** compilation the practical default, even in dev.

Why it mattered for my migration: Ivy was the highest-risk hop because **libraries built for View Engine** had to be made Ivy-compatible. Angular shipped `ngcc` (**Angular Compatibility Compiler**) to transform View-Engine library builds into Ivy format at install time, which bridged most libs — but any library that was truly incompatible had to be upgraded, replaced, or isolated. Understanding *why* Ivy could break a library (it consumes different compiled metadata) is what let me triage each dependency instead of guessing.

AOT vs JIT (the natural sub-follow-up): **JIT** compiles templates in the browser at runtime (bigger, slower start, ships the compiler); **AOT** compiles at build time (smaller, faster start, template errors caught at build, no compiler shipped). Ivy made AOT cheap enough to be the default, which is a big reason builds got faster.

Q2. How does Angular change detection work?

Answer. Angular keeps a **component tree**. When something *might* have changed, it runs a **dirty check** top-down over that tree, comparing each binding's current value to its previous value and updating the DOM where they differ.

- **What triggers a check?** Asynchronous events — DOM events, timers (`setTimeout` / `setInterval`), and XHR/`fetch`. Angular knows about them because of **zone.js** (next question).

- **Default strategy** checks the whole tree on every trigger — simple, but can be wasteful on large trees.
- **OnPush** narrows checking: a component is only checked when an `@Input` **reference changes**, an **event fires inside it**, an **observable it uses via `async` emits**, or you explicitly `markForCheck()`. Combined with immutable data and the `async` pipe, that cuts redundant checks.

Why it mattered for my migration: Ivy subtly changed change-detection timing in a few cases (e.g. when queries resolve, expression-changed-after-checked situations). A handful of components that "happened to work" under View Engine surfaced `ExpressionChangedAfterItHasBeenCheckedError` or timing issues under Ivy, and I had to understand the CD lifecycle to fix them properly (move the update into the right lifecycle hook, or trigger detection deliberately) rather than paper over them.

Q3. What is zone.js and why does Angular use it?

Answer. zone.js monkey-patches asynchronous browser APIs (`addEventListener`, `setTimeout`, `Promise`, XHR, etc.). By wrapping them, it can tell Angular "an async task just finished — something may have changed, run change detection." That's the "magic" that lets you mutate a field in a click handler and see the view update without telling Angular anything.

- **Trade-off:** zone.js adds bundle weight and patches globals, and it triggers CD even when nothing relevant changed.
- **Where it's going: zoneless** change detection (stable in Angular 19; experimental earlier) drops the zone.js polyfill and relies on **signals**, the `async` pipe, and explicit `markForCheck()` instead — smaller and more predictable.

Why it mattered for my migration: each major pins a compatible **zone.js** version, so `ng update` bumped it per step — I had to keep it in the compatible range. And landing on Angular 16 with `takeUntilDestroyed()`/signals available sets up a *future* zoneless move, which is the honest "here's our reality + modern direction" story.

Q4. Which RxJS operators/behaviors changed, and how did that surface in the migration?

Answer. The big shifts were **RxJS 5→6** (v6) and **6→7** (v12):

- **5→6:** operators became **pipeable** (`.pipe(map(), filter())`) instead of prototype-patched (`obs.map().filter()`), with new import paths (`rxjs`, `rxjs/operators`). This touched every stream in the codebase.
- **6→7:** `toPromise()` **deprecated** → `firstValueFrom` / `lastValueFrom`; **stricter typing** on operator overloads (`combineLatest`, `merge`); some subtle completion-semantics tightening.

Operators I actually rely on and why (the interviewer will ask): - `switchMap` — cancel the previous inner stream when a new value arrives ("latest wins": search, token refresh). - `mergeMap` — run inner streams in parallel (independent writes). - `concatMap` — queue inner streams in order (order matters). - `combineLatest` — derive a value from the latest of several streams. - `debounceTime` — rate-limit rapid

input. - `catchError` — recover or rethrow inside a stream. - `takeUntil` — complete a subscription on a `destroy$` signal (leak prevention).

Why it mattered for my migration: the RxJS changes were the most *pervasive* mechanical edits — they weren't in one file, they were everywhere a stream existed. Understanding the operator semantics is what let me convert them correctly instead of blindly find-replacing (e.g. knowing `firstValueFrom` throws on empty completion whereas `toPromise` resolved `undefined` — a real behavior difference).

Q5. Explain the Angular lifecycle hooks — and did the migration touch them?

Answer. In order: - `ngOnChanges` — on `@Input` changes (before first `ngOnInit`). - `ngOnInit` — once, after first inputs set — init logic, initial data fetch. - `ngDoCheck` — every CD run (custom change detection). - `ngAfterContentInit` / `ngAfterContentChecked` — projected content ready/checked. - `ngAfterViewInit` / `ngAfterViewChecked` — view & child views ready/checked — safe to read `@ViewChild`. - `ngOnDestroy` — cleanup: unsubscribe, clear timers.

Why it mattered for my migration: the `ViewChild` **static flag** (v8/9) is *directly* about lifecycle timing — `static: true` resolves the query before CD so it's available in `ngOnInit`; `static: false` resolves after, available in `ngAfterViewInit`. Getting that wrong gives you an `undefined` ref. And the Ivy CD-timing fixes were about moving work into the correct hook. So the lifecycle wasn't academic — it's the exact mechanism behind two of my breaking changes.

Q6. NgModules vs standalone components — where did you land?

Answer. - **NgModules** (`@NgModule`) are the classic packaging unit: declarations, imports, providers, exports. Every component belongs to exactly one module. - **Standalone components** (preview v14, **stable v15**) drop the module — a component declares its own `imports` directly (`standalone: true`), so you can bootstrap and route without `NgModule` boilerplate. This is the modern default direction and pairs with `provideRouter`, `provideHttpClient`, etc.

Where we landed (honest): the apps finished the migration as **NgModule apps on Angular 16** — the goal was "safely on a supported version, nothing regressed," not a re-architecture. I deliberately **did not** rewrite to standalone mid-migration because that's a second, separate risk. My recommended next step was: adopt **standalone components**, **signals**, and `takeUntilDestroyed()` incrementally on Angular 16, since 16 already ships all three.

That's the strong answer: I know the modern architecture, I can articulate the migration path to it, and I made a *disciplined* call not to conflate two migrations.

Q7. What made the bundles smaller / builds faster after the migration? (the payoff)

Answer. Concretely: - **Ivy's tree-shaking + AOT-by-default** → smaller production bundles and faster builds. - **Newer TypeScript** per major → faster type-checking and better incremental compilation. - **Angular 16's esbuild-based dev server** (preview) → dramatically faster dev rebuilds vs the old webpack pipeline. - **Dropping IE11** (v13) → no legacy/differential bundle overhead.

So "modern, supported, faster-building apps on one Angular version" isn't a slogan — it maps to specific mechanisms: Ivy, AOT, newer TS, esbuild, and shedding legacy targets.

04 — Testing Strategy, STAR Stories, Git Workflow

The migration's whole promise was "*nothing regressed*." That promise is only as good as the testing behind it. This file is the testing strategy, the STAR stories interviewers ask for by name, the Git workflow, and "what I'd do differently."

Part A — The seamless-transition testing strategy

Q1. How did you make the transition "seamless" — what was the testing strategy?

Answer. Testing was layered around the per-step loop, not bolted on at the end:

1. **Build + run the full unit suite after every version hop** — not just at the end. This localizes any failure to a single major's changes.
2. **Smoke-test the critical flows** in each app after each hop — for Ticketing that's create/assign/resolve a ticket; for the plugins it's mount + core interaction inside the host.
3. **Add coverage to the riskiest areas before upgrading them** — where the existing tests were thin, I wrote tests first so I'd actually *notice* a regression instead of hoping.
4. **QA full regression** on the Ticketing app and the three plugin apps — QA validated each app end-to-end; I fixed what they found.
5. **Integration-boundary testing for the SSG plugins** — verifying the bootstrap/mount point inside the host, since that's where an embedded plugin breaks in ways unit tests can't see.
6. **One commit per version** so any regression was attributable and revertible.

The phrase I use: "*Seamless doesn't mean lucky — it means every hop was gated by build + tests + smoke, and QA regression on top.*"

Q2. What tooling did you test with?

Answer. **Jasmine + Karma** for unit/component tests (the Backoffice apps' existing setup). I ran the suite via `ng test` at each step. For the plugins I leaned harder on **manual smoke + QA integration testing** because the meaningful risk was the *host integration*, which is awkward to cover in Karma. I also treated the **build itself as a test** — strict templates and the newer TS compiler catch a whole class of regressions before any spec runs.

Q3. Why add tests *before* upgrading rather than *after*?

Answer. Because a test written *after* a migration only proves the *new* behavior — it can't catch a regression you already introduced. A test written *before* captures the *known-good* behavior, so when the upgrade changes it, the test fails and tells you. For the riskiest, least-covered areas I wrote characterization tests first, then upgraded, then watched them.

Part B — STAR stories (memorize the shape, tell them naturally)

STAR 1 — The RxJS 6 → 7 migration

- **Situation:** Around the Angular 11→12 hops, the apps moved to RxJS 7. `toPromise()` was deprecated and operator overloads got stricter — and streams were used *everywhere* in the codebases.
- **Task:** Migrate all RxJS usage without changing behavior, across four repos.
- **Action:** I ran the RxJS migration schematic, then did the manual work it couldn't: replaced every `toPromise()` with `firstValueFrom / lastValueFrom`, and added explicit types where the stricter overloads flagged loosely-typed `map / reduce` chains. I was careful about a real semantic difference — `firstValueFrom` *throws* on empty completion whereas `toPromise` resolved `undefined` — so I checked each call site rather than blind find-replace.
- **Result:** All streams on RxJS 7, no behavior change, build and tests green. The edits were mechanical but had to be precise; doing them one repo at a time meant I'd solved the pattern once and applied it fast to the rest.

```
// before (RxJS 6, deprecated) — resolves undefined on empty stream
const data = await this.http.get('/api/tickets').toPromise();

// after (RxJS 7) — throws on empty; verified each call site expects a value
import { firstValueFrom } from 'rxjs';
const data = await firstValueFrom(this.http.get('/api/tickets'));
```

STAR 2 — The Ivy-incompatible third-party library

- **Situation:** At Angular 9, Ivy became the default renderer. One of the apps depended on a library built for View Engine that wasn't cleanly Ivy-compatible.
- **Task:** Get past the v9 hop without that library blocking the whole migration or breaking the app.
- **Action:** I checked each dependency's Angular/Ivy compatibility *before* the hop and built a small compatibility matrix. For the blocking library I evaluated three options in order: **(1)** upgrade it to a version that shipped Ivy support; **(2)** swap it for a maintained, Ivy-compatible alternative; **(3)** if neither existed yet, pin the current version and isolate it so it didn't hold up the rest of the step. Angular's `ngcc` bridged most View-Engine libs automatically, so the truly-blocking cases were few.
- **Result:** Cleared v9 with all dependencies Ivy-compatible. By the time I reached v13 (where View Engine was fully removed) there was nothing left to fix, because I'd triaged every library at v9 rather than deferring it.

STAR 3 — SSG plugins loading inside a host environment

- **Situation:** Three of the four repos weren't standalone apps — they were **SSG web plugins embedded and bootstrapped inside a host environment**. An upgrade that works in isolation can still fail when mounted in the host.
- **Task:** Upgrade the plugins to Angular 16 and guarantee they still load and behave correctly *inside* the host, not just on their own.

- **Action:** I treated the **integration boundary** — how the plugin is bootstrapped/mounted and how its Angular runtime coexists with the host — as a first-class test target, as important as in-app behavior. After each hop I verified the built plugin bundle still mounted cleanly in the host, didn't clash with the host's runtime, and that the critical in-plugin flows worked *in that context*. QA regression-tested the plugins inside the real host.
- **Result:** All three plugins upgraded to 16 and mounting correctly in the host, no runtime clashes. The lesson I state: for embedded/micro-frontend-style code, "it builds and unit-tests pass" isn't done — the integration boundary is where the real risk lives.

STAR 4 — "How did you make a multi-version migration seamless / lead it across four repos?"

- **Situation:** Four repos on Angular 2, 4, and 9 needed to reach 16 without disrupting the feature teams that owned them.
- **Task:** Own the migrations, apply the cross-version breaking changes consistently, and hand back working apps.
- **Action:** I built one **playbook** — never jump majors, `ng update` per step, `update.angular.io` checklist, build+test+smoke each hop, one commit per version — and reused it across all four repos, sequentially so later repos benefited from fixes I'd already solved. I coordinated with each owning team on PR review and with QA on regression scope.
- **Result:** Four repos on a single, modern, supported Angular 16 with faster builds; QA-validated; and a granular commit history that documented exactly what each version required. I was the person who understood the cross-version changes and applied them consistently — which is exactly why it went smoothly.

Part C — Git workflow

Q4. How did you manage this in Git?

Answer. - **A branch per repo** for the migration. - **One commit per major version step**, with a clear message ("upgrade to Angular 13: remove View Engine refs, drop entryComponents"). - **PRs reviewed by the owning feature team** before merge.

The granular commits paid off three ways: `git bisect` to find which version introduced a regression, **one-step rollback** to the previous version if a hop went wrong, and **free documentation** — the history *is* the record of what each version required.

```
* chore: upgrade to Angular 16
* chore: upgrade to Angular 15
* chore: upgrade to Angular 14
* chore: upgrade to Angular 13 (remove View Engine, drop entryComponents)
* chore: upgrade to Angular 12 (RxJS 7, strict mode)
* ...
(base branch)
```

Part D — Reflection

Q5. What would you do differently?

Answer. Two things: 1. **Strengthen automated tests *before* starting.** Strong coverage is what makes a big upgrade fast and safe; where tests were thin I added them mid-flight, and I'd rather have that cushion in place up front. 2. **Script the per-step build/test loop in CI** so every version bump is *gated automatically* — build, unit tests, and lint green as a merge condition — instead of me verifying each hop by hand. That turns the discipline into infrastructure so it can't be skipped under time pressure.

Q6. What did this project prove about you?

Answer. That I understand Angular deeply enough to move a codebase across a decade of breaking changes without regressions — Ivy, RxJS's evolution, change detection, the build pipeline — and that I do risky work with **discipline**: incremental steps, gated checkpoints, attributable commits, and a rollback plan. It was a focused migration, but it's exactly the kind of deep-internals + risk-management work a senior Angular engineer should own.

Rapid-fire answers (drill these out loud)

- **"Why not just go straight to 16?"** — Migration schematics are per-step; a big jump loses all automation and makes every failure unattributable.
- **"What broke at v9?"** — Ivy became default; View-Engine libraries had to be made Ivy-compatible; removed `entryComponents`.
- **"toPromise replacement?"** — `firstValueFrom` / `lastValueFrom`.
- **"How did you test?"** — Build + unit tests + smoke every hop, coverage on risky areas first, QA full regression, integration-boundary tests for the plugins.
- **"How did you roll back?"** — One commit per version; revert one hop; app stays shippable on the prior major.
- **"Plugins — what was special?"** — Embedded in a host SSG environment; verifying bootstrap/mount + no runtime clash mattered as much as in-app behavior.
- **"Where did you land?"** — NgModule apps on Angular 16; standalone + signals + `takeUntilDestroyed()` flagged as the next step.